



momently

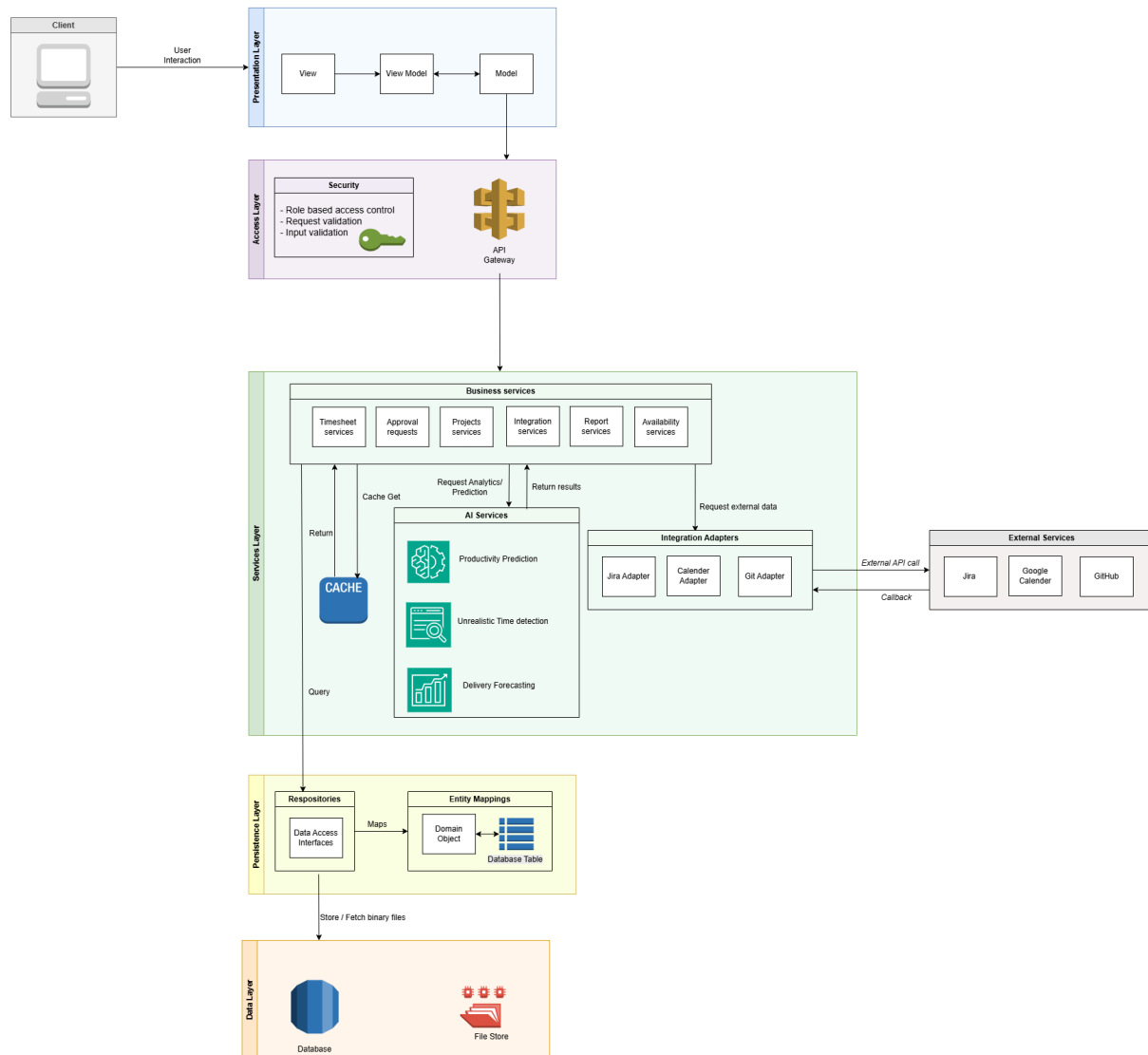
Architectural Patterns



CYBERNAUTS

Software Architecture Diagram

5-Layer Software Architecture Diagram



Architectural Patterns

The Momently system adopts a combination of modern, modular, and scalable architectural patterns to ensure maintainability, performance, security, and adaptability for future growth.

5-Layered Architecture

The system is structured using a 5-layered architecture that organizes core application logic into horizontal, decoupled tiers. This strict separation of concerns improves maintainability, testability, and scalability.

Although the system operates as a structured monolithic client-server application, internally it is composed of modular subsystems that interact through well-defined interfaces.

Overview of Subsystems

Momently is composed of five primary subsystems, each responsible for a distinct domain:

- Time Tracking Subsystem
- Timesheet & Approval Subsystem
- Reporting & Analytics Subsystem
- Integration Subsystem
- Identity & Access Subsystem

Benefits & Non-Functional Requirements (NFR) Alignment

Security

Security is enforced through layered controls including JWT authentication, Role-Based Access Control (RBAC), and strict input validation at system boundaries.

The Angular frontend is secured via Nginx routing, while the Spring Boot backend functions as the primary API gateway and security enforcement layer. The FastAPI AI service is isolated within a private Docker network, ensuring it is not directly accessible from the public internet. This architecture limits threat exposure and prevents external actors from directly accessing sensitive services such as the AI engine or PostgreSQL database.

Scalability

The system is designed to handle high-load scenarios by offloading resource-intensive operations away from the main application flow.

Redis is used as a high-performance caching layer and/or message broker, enabling the system to efficiently manage bursts of activity, reduce database load, and support high-frequency operations such as time entry logging without degrading API performance.

Reliability & Availability

The containerized and modular architecture reduces inter-service dependencies and eliminates single points of failure.

Because subsystems are decoupled, failures in non-critical services (such as the FastAPI AI service) do not affect core business functionality. For example, if the AI service becomes unavailable, users can still log time, manage tasks, and access core features through the Spring Boot backend without disruption.

Compatibility & Evolvability

The system's modular structure enables independent evolution of components without requiring system-wide changes.

Each layer and service can be updated, replaced, or scaled independently, allowing the platform to adopt new technologies or improvements without impacting unrelated components.

Layered Architecture

The system is divided into five layers, each responsible for a specific aspect of application processing:

Presentation Layer

The Presentation Layer is implemented using Angular and represents the user-facing interface of the system. It follows the MVVM (Model-View-ViewModel) pattern, where user interactions in the View are automatically reflected in the ViewModel through data binding. The ViewModel then structures and sends HTTP requests to the backend services.

Access Layer

The Access Layer functions as the system's security gateway. It is typically implemented using an API gateway or Spring Security proxy. This layer is responsible for authenticating incoming requests, enforcing authorization rules through RBAC, and validating inputs to protect against malicious or invalid data.

Services Layer

The Services Layer contains the core business logic of the system. It handles operations such as timesheet processing, approvals, reporting, and external integrations.

This layer also incorporates supporting services such as:

- Redis for caching and performance optimization
- FastAPI-based AI services for advanced analytics (e.g. productivity prediction and forecasting)
- Integration adapters for external tools such as Jira, Google Calendar, and GitHub

Persistence Layer

The Persistence Layer manages data access and transformation. It uses Spring Data JPA/Hibernate to map application objects to relational database structures through repository abstractions.

Data Layer

The Data Layer represents the physical storage infrastructure. It is deployed within a Docker environment and uses PostgreSQL as the primary relational database for structured data storage.

Caching Strategy

Caching plays a critical role in improving system performance and reducing database load. The system checks the cache before querying the database for read operations.

- On a cache hit, data is returned directly from Redis.
- On a cache miss, data is retrieved from the database, stored in the cache, and then returned to the client.

This approach reduces database pressure, improves response times, and enhances overall system scalability.

Frequently Cached Data Examples

- User session data
- Dashboard and summary metrics
- External API responses (e.g. Jira, Google Calendar, GitHub)
- Frequently accessed reference data (e.g. active projects, workspace users)
- Productivity scores (e.g. weekly summaries)

MVVM Architecture

The frontend follows the MVVM (Model-View-ViewModel) architectural pattern, which separates concerns into three distinct layers:

- Model: Represents raw application data
- View: The user interface
- ViewModel: Acts as an intermediary that manages application state and logic

The ViewModel transforms raw data into a format suitable for presentation and handles communication between the View and backend services. This ensures a clear separation between UI logic and business/data logic, improving maintainability and testability.

Design Patterns

Strategy Pattern

External integrations such as Jira, GitHub, calendars. Each has different APIs but serves similar purposes. A common interface for fetching tasks or logging work would allow the system to swap implementations without changing business logic. Adding a new integration becomes a matter of writing one new strategy class. This ensures that the system does not get affected by impacts made to these external tools such as API updates or downtime. If Jira changes its API tomorrow, only the Jira strategy needs updating. The timer service that logs hours against a ticket doesn't change at all.

Chain of Responsibility and Pipe & Filters

Each incoming HTTP request passes through a sequence of filters before reaching the actual controller. Each filter has its own responsibility and they all do handle security, authorisation and authentication without knowing each other. Such as JWT validation, CORS handling, CSRF protection, and authorization. The JWT filter doesn't care what the CORS filter does. It just validates the token and passes the request along. If a request fails any filter, it never reaches the controller.

Observer Pattern

When a timesheet gets submitted, several things should happen: notify the manager, update analytics, trigger AI anomaly detection and even send a notification. An event publisher could notify all interested listeners without the timesheet service knowing about each one individually.

Proxy Pattern for Caching

A caching layer could sit between the service and repository. The proxy would check Redis before hitting the database, returning cached results when available. The service wouldn't need to know caching exists. This keeps caching logic separate and makes it easy to adjust cache expiration or disable caching entirely without touching service code.

Adapter Pattern

External APIs change. A Jira adapter could translate between the system's internal task format and Jira's specific API. When Jira changes, only the adapter needs updating. The rest of the system stays the same. We will be integrating tools such as Jira, Google Calender or GitHub.

Technology Requirements

Frontend Framework

The frontend handles time tracking, analytics, and AI-driven insights, so it needs to stay responsive and accessible. Real-time updates and a modular component architecture are essential because users expect to see their active timer update instantly. Furthermore, the interface should feel efficient even when displaying complex reports or AI-generated recommendations.

Technologies Considered

Angular 17 (Selected)

A TypeScript-based frontend framework developed by Google that provides built-in support for dependency injection, routing, and reactive form handling.

Pros:

- The component-based structure keeps the code organised and easier to maintain, with clear separation of concerns
- Built-in features like routing, form handling, and change detection make it well-suited for complex user interfaces
- TypeScript helps catch errors early, improving overall code quality and maintainability
- Angular Material provides a ready-made, professional UI component library that speeds up development
- Testing tools like Jasmine and Karma are included by default, so there's no need to set up external libraries
- It's widely used in enterprise environments, which shows its reliability and long-term stability

Cons:

- Has a steeper learning curve than React
- Requires more setup code for simple components, which can slow down quick development

React

A JavaScript library built around a component-driven approach, supported by a large and active ecosystem.

Pros:

- Has a huge community with a wide range of libraries and tools available
- Very flexible and lightweight to work with
- JSX helps make UI structure easier to read and understand

Cons:

- It's less opinionated, so the team has to make more architectural decisions
- Routing and state management usually require extra libraries
- The flexibility can sometimes lead to inconsistent coding patterns across a team

Vue.js

Overview:

A progressive framework that is easy to learn and can be introduced gradually into projects.

Pros:

- Easy to pick up, especially for new developers
- Well-documented and beginner-friendly
- Flexible and easy to integrate into existing projects

Cons:

- Less commonly used in large enterprise systems compared to Angular or React
- Not as naturally structured for large-scale applications without additional conventions
- Smaller talent pool in enterprise environments
- Final Selection: Angular 17 + Angular Material

Final Selection: Angular 17+

Angular was chosen because its structured, component-based architecture aligns well with the complexity of the system. Features like time tracking, analytics, and multiple external integrations benefit from a clear separation of concerns. Its built-in routing, form handling, and change detection also help maintain fast and responsive interactions, supporting the target 1–2 second response time without needing extra libraries.

Angular enforces consistent development patterns and structure, which helps reduce inconsistency and improves maintainability. Angular Material further supports this by providing a clean, ready-to-use component library that speeds up development. Overall, Angular's strong TypeScript integration, built-in testing support, and enterprise-focused design make it a reliable choice for a large-scale, business-critical application.

Backend Framework & Service Layer

The backend handles authentication, time tracking, project management, timesheet approvals, and external integrations. Security, modularity, and performance are top priorities.

Technologies Considered

Spring Boot 3.5+ with Java 17 (Selected)

Spring Boot is a production-ready framework used to build stand-alone Java applications with embedded servers and minimal configuration. It is widely used for enterprise systems because it provides a structured and scalable development approach.

Pros:

- Built-in support for developing REST APIs and integration testing
- Works naturally with hexagonal and layered architectures
- Java 17 LTS provides long-term stability, security updates, and future compatibility
- Strong encapsulation improves security by reducing unauthorized access risks
- Comprehensive ecosystem including Spring Security, Spring Data JPA, and caching support

- Reliable performance and scalability for enterprise-level applications

Cons:

- Has a steeper learning curve compared to frameworks like Express.js
- Slower startup time during development
- Some manual configuration is still required despite auto-configuration features

Express.js + Node.js

Express.js is a lightweight and flexible framework used for building REST APIs with JavaScript or TypeScript.

Pros:

- Quick and simple to set up
- Large npm ecosystem with many available libraries
- Allows the frontend and backend to use the same language

Cons:

- Provides fewer built-in architectural patterns for large-scale systems
- More difficult to manage complex business logic as the system grows
- Requires more manual handling for security, validation, and transaction management

Final Selection: Spring Boot 3.5+ with Java 17

After evaluating the different backend options, Spring Boot with Java 17 was selected because it provides the best balance of scalability, security, maintainability, and long-term support for a business-critical application.

One of the main advantages is that Spring Boot already includes strong support for REST API development, integration testing, and structured application design, which aligns well with the hexagonal and layered architecture being used in the system.

Java 17 LTS also offers long-term stability, regular security updates, and future compatibility, making it a reliable choice for enterprise development. The framework's strong encapsulation and mature security features help reduce the risk of unauthorized access and support the requirement for secure RESTful APIs. Compared to alternatives such as Express.js, which can become difficult to structure in large systems, or Django, which introduces a separate technology stack, Spring Boot provides a more cohesive solution for the team's skills and project requirements.

Database Management System

The system needs a database that can reliably store and manage time entries, project data, user information, and audit logs. Because this data is highly structured and consistency is critical, the database must support strong transactional guarantees and strict access control.

Technologies Considered

PostgreSQL (Client-Required)

PostgreSQL is an advanced open-source relational database that fully supports ACID transactions, strong consistency, and complex querying.

Pros:

- Ensures strong consistency and referential integrity through foreign keys and transactions
- Supports advanced querying features like CTEs and materialized views, which are useful for reporting and analytics
- JSONB support allows flexibility when data structures evolve over time
- Includes strong security features such as row-level security and role-based access control, which is useful for multi-tenant or permission-heavy systems
- Already required by the client, making it the default choice

Cons:

- Schema changes require careful planning and proper migration strategies
- Not as naturally suited as NoSQL databases for highly dynamic or schema-less data

MySQL

MySQL is a widely used relational database known for performance and simplicity.

Pros:

- Very fast read performance in many workloads
- Large community and strong industry adoption
- Proven stability in enterprise systems

Cons:

- Weaker support for advanced JSON features compared to PostgreSQL
- Fewer built-in analytical capabilities
- Not aligned with the client's specified database requirement

MongoDB

MongoDB is a document-based NoSQL database designed for flexible schemas and horizontal scaling.

Pros:

- Flexible schema makes it easy to iterate quickly during development
- Scales horizontally very well for large distributed systems

Cons:

- Does not enforce relational integrity like traditional SQL databases
- Weaker support for complex transactions
- Not suitable for systems that require strict consistency and structured relationships, such as this one

Final Selection: PostgreSQL

It handles structured relational data very effectively and guarantees transactional integrity, which is essential for features like time tracking and audit logging where accuracy is critical. Its strong indexing and query optimization also make it well-suited for reporting and analytics use cases.

While NoSQL solutions like MongoDB offer flexibility, they trade off consistency and relational integrity, which are important for this system. Overall, PostgreSQL provides the best balance of reliability, performance, and enterprise-level features needed for this application.

Caching Layer

Caching is used to improve performance by temporarily storing frequently accessed data such as API responses, session data, and commonly requested database results.

Technology Selected: Redis

Redis was chosen because it is specifically designed for fast, in-memory data access, which makes it ideal for caching.

In this system, Redis is used to store things like responses from external services (for example Jira and GitHub data) and user session information. This reduces the need to repeatedly call external APIs or query the database, which significantly improves response times and helps the system meet the target of 1–2 second response times.

Compared to relying on a relational database like PostgreSQL for repeated reads, Redis is much faster because it operates entirely in memory. It also supports features like Time-to-Live (TTL), which makes it easy to automatically expire outdated cache entries and keep data fresh without manual cleanup.

API Communication

Technology Selected: RESTful APIs

RESTful APIs are used as the main way for the frontend and backend to communicate. They're a good fit here because they're a widely accepted standard and already align with the client's requirements. Using REST also keeps the frontend and backend clearly separated, which means each part can be developed, scaled, and maintained independently without tightly depending on the other.

Another benefit is that REST makes it straightforward to integrate external services like Jira, GitHub, and calendar systems, since most modern APIs already follow REST principles. On the backend, Spring Boot makes this even easier by providing strong built-in support for building REST controllers, validating requests and responses, and handling errors in a clean and consistent way. This helps keep the API design structured and aligned with enterprise development practices.

Hosting & Deployment Platform

The application runs on a scalable, secure platform with disaster recovery and high availability.

Technology Selected: AWS (Amazon Web Services)

The application is deployed on AWS, which provides a scalable and secure cloud environment with built-in support for high availability and disaster recovery. AWS was chosen partly because the client already provides access to AWS resources, which makes it the most practical option for the project. This reduces setup effort and allows the system to be built directly on existing infrastructure.

Another major advantage is that AWS supports the system's reliability requirements, including high availability and the ability to recover within a two-hour window in the event of a failure. It also scales easily as usage grows, which is important for a production system.

AWS integrates well with other parts of the architecture too, including CI/CD pipelines, monitoring tools, and managed services like RDS for PostgreSQL and ElastiCache for Redis. This helps keep the infrastructure consistent and easier to manage. While Microsoft Azure was also considered, AWS was selected because it is already available for the project and is widely adopted in enterprise and financial environments.

Authentication & Authorization

The system needs secure, stateless authentication and fine-grained access control for users, roles, and workspaces.

Technology Selected: Spring Security + JWT (JSON Web Tokens)

The system needs a secure way to handle user login, access control, and permissions across different roles and workspaces. To achieve this, we use Spring Security together with JWT. Spring Security takes care of the core authentication and authorization logic. It allows us to define clear role-based access control, such as Admin, Manager, and Developer roles, and ensures users can only access what they are allowed to.

JWT is used to make the system stateless. Instead of storing user sessions on the server, authentication information is encoded into a token that is sent with each request. This makes the system easier to scale because any backend instance can validate the token without relying on shared session storage.

Another advantage is that JWTs can include useful information like user roles and workspace memberships. This allows the system to make authorization decisions quickly without needing extra database queries every time.

Overall, this approach keeps the system secure, scalable, and flexible, while also leaving room for future integration with standards like OAuth2 or OpenID Connect if needed.

AI and Machine Learning

The system supports predictive analytics, anomaly detection, and AI-driven insights for time tracking and productivity reporting.

Technology Selected: PyTorch + TensorFlow (Python Microservices)

PyTorch and TensorFlow are used to power the AI functionality, which runs in separate Python-based microservices.

This separation is intentional — it keeps the AI workload independent from the main application. That means predictive models and analysis can scale on their own without impacting the performance of the core system.

PyTorch is particularly useful during model development and experimentation, especially for tasks like anomaly detection where models may need frequent tuning and iteration. TensorFlow, on the other hand, is well-suited for deploying models into production environments where stability and performance are more important.

Using Python microservices also makes sense because Python has a very strong ecosystem for machine learning and data science. It allows faster development and gives access to a wider range of AI/ML tools compared to implementing similar functionality directly in Java.

Overall, this approach keeps the AI layer flexible, scalable, and well-suited for continuous improvement while staying decoupled from the main backend system.

Testing Strategies

Testing happens at multiple levels to ensure system reliability.

Backend Testing

Tool	Purpose
JUnit 5	Core testing framework for unit and integration tests
Mockito	Mocks dependencies to isolate components during unit testing
Spring Boot Test	Handles integration testing for database interactions, REST endpoints, and context loading
Testcontainers	Runs integration tests with real database instances
AssertJ	Provides fluent assertions for readable test code

Frontend Testing

Tool	Purpose
Cypress	End-to-end testing that simulates user interactions
Angular Testing Utilities	Component integration testing
Jest	

AI/ML Testing

Tool	Purpose
PyTest	Testing framework for Python-based AI services
Model Validation	Tests model accuracy against benchmarks
Preprocessing Validation	Verifies data transformation pipelines
Endpoint Testing	Ensures prediction APIs return expected responses

This testing strategy improves system reliability by thoroughly validating both core logic and full system integration. JUnit 5 and Mockito are used for isolated unit testing of backend components, while Spring Boot Test and Testcontainers ensure that database interactions, REST endpoints, and application integration behave correctly in realistic environments.

For the frontend, Jest and Angular Testing Utilities are used for component and integration testing, while Cypress validates complete user workflows by simulating real user interactions. In the AI/ML layer, PyTest, model validation, preprocessing validation, and endpoint testing ensure prediction accuracy, reliable data transformation, and stable API responses. Together, these testing approaches help ensure the system is stable, maintainable, and performs reliably across all layers of the application.

External Integrations

The system integrates with Jira, GitHub, and calendar services for time synchronization and data comparison.

Technology Selected: Spring WebClient

Spring WebClient handles integrations with GitHub, Jira, and calendar systems. It supports non-blocking, asynchronous communication, which helps the system handle multiple external API calls without degrading performance.

This approach meets the requirement of comparing tracked time against external data sources (Jira tickets, GitHub issues, calendar events). WebClient's reactive nature ensures efficient resource utilization, which matters especially when dealing with rate-limited external APIs.

CD and Version Control

Building, testing, and deployment happen automatically through a structured workflow.

Technology Selected: Git + GitHub + GitHub Actions

Git and GitHub handle version control, enabling efficient collaboration, code management, and issue tracking. GitHub Projects manages the development process using Scrum methodology, with weekly sprints, regular reviews, and stand-ups to maintain consistent progress.

GitHub Actions implements CI/CD pipelines, automating the build, test, and deployment process. Code pushed to the dev branch deploys to the development environment, while code merged to main triggers production deployment.

This aligns with the requirement for a structured development workflow and ensures consistently delivered working code while reducing human error. Environment-specific configurations are managed through Spring Profiles and GitHub secrets.

Branching strategy

Feature branching was used.

Branch	Purpose	Deployment Target
feature/*	Feature development	Local
dev	Integration branch	Development environment
main	Production-ready code	Production environment

Summary Technology Stack

Layer	Technology
Frontend	Angular 17, Material UI
Backend	Spring Boot 3.5+, Java 17
Database	PostgreSQL
Caching	Redis
Containerization	Docker, Docker Compose
Hosting	AWS (EC2, RDS, ElastiCache, Parameter Store)
Authentication	Spring Security, JWT
AI/ML	PyTorch, TensorFlow (Python microservices)
Testing	JUnit 5, Mockito, Spring Boot Test, Cypress, PyTest
API Communication	RESTful APIs, Swagger / OpenAPI
External Integrations	Spring WebClient
CI/CD	Git, GitHub, GitHub Actions
Monitoring	Prometheus, Grafana
Methodology	Scrum with GitHub Projects